

"I need to evaluate the results of my AI term project. My model outputs multi-class labels for a six category waste classification of cardboard, glass, metal, paper, plastic, trash and I am comparing it against my naive baseline that picks the majority class of paper with a 23.67% accuracy. My trained CNN PyTorch model has a ~75% accuracy. YOUR INSTRUCTIONS: 1. Suggest the most appropriate statistical significance test for my specific data type (e.g., 5x2cv paired t-test, Wilcoxon Signed-Rank, McNemar's). 2. Detail exactly what metrics (e.g., F1, RMSE, LLM-as-a-Judge JSON rubrics) I should visualize to prove my claims. 3. Act

Show more ▾

For your setup — **single-label multi-class classification** with a **naive majority-class baseline** vs a **trained CNN** — the strongest evaluation framing is:

- compare **paired predictions on the same test examples**
- report **class-sensitive metrics** (because your classes are imbalanced)
- show **error structure**, not just top-line accuracy

1. Best Statistical Significance Test

Primary recommendation: McNemar's Test

Use:

McNemar's test

Why it fits your case:

- You are comparing **two classifiers**
- On the **same test set**
- With **categorical predictions**
- And you care whether one model is significantly more accurate than the other

This is exactly what McNemar's test was designed for.

Your baseline and CNN produce paired outcomes per image:

Example	Baseline Correct?	CNN Correct?
1	0	1
2	1	1
3	0	0

McNemar specifically tests whether the disagreement counts are asymmetric.

Important caveat

McNemar is naturally binary (correct/incorrect), but that is fine here because your hypothesis is:

“The CNN is more accurate than the baseline.”

You reduce each prediction to:

- correct
- incorrect

and compare paired correctness.

Why NOT 5x2cv paired t-test?

5x2 cross-validated paired t-test

This is better when:

- comparing learning algorithms across repeated train/test splits
- evaluating instability due to data partitioning

But reviewers often dislike it when:

- only one dataset exists
- the same trained model is reused
- folds are not fully independent

For a class project, it is probably unnecessary unless you performed repeated retraining.

Why NOT Wilcoxon Signed-Rank?

Wilcoxon signed-rank test

Wilcoxon is appropriate when:

- you have per-fold metric scores
- such as 10 F1 values from 10 runs

Example:

- Run 1 macro-F1
- Run 2 macro-F1
- etc.

If you trained the CNN multiple times with different random seeds or folds, then:

- Wilcoxon on per-run macro-F1 scores becomes excellent.

Without repeated runs, McNemar is stronger.

Recommended Evaluation Setup

If you only trained once

Use:

- McNemar's test
- confusion matrix
- macro-F1
- weighted-F1
- per-class recall

If you trained multiple times/folds

Use:

- Wilcoxon signed-rank on macro-F1 or accuracy
- report mean $\pm$ std
- optionally add confidence intervals

2. Metrics & Visualizations You SHOULD Show

Accuracy alone is insufficient because your baseline exploits class imbalance.

A reviewer will immediately ask:

“Does the CNN actually learn minority classes?”

You need metrics proving that.

Essential Metrics

A. Confusion Matrix (MOST IMPORTANT)

Use normalized confusion matrices.

This shows:

- what classes get confused
- whether “trash” collapses into “paper”
- whether minority classes are learned

You should include:

1. Baseline confusion matrix
2. CNN confusion matrix

The difference will visually demonstrate learning.

For example:

- baseline predicts almost everything as paper
- CNN distributes predictions meaningfully

This is one of the strongest figures in your paper.

B. Macro F1 Score

Use:

F1 score

Specifically:

- macro-F1

Why:

- treats each class equally
- punishes majority-class bias
- ideal for imbalanced multi-class datasets

This is probably your most important scalar metric.

A reviewer may value macro-F1 more than accuracy.

C. Per-Class Precision / Recall / F1

Show a table like:

Class	Precision	Recall	F1
cardboard	...	...	...
glass	...	...	...

This proves:

- the model works across categories
- not only on paper

D. Balanced Accuracy

Use:

Balanced accuracy

This compensates for class imbalance.

Your naive baseline likely has:

- terrible balanced accuracy
- despite 23.67% raw accuracy

This strengthens your argument.

E. ROC-AUC (Optional)

Multi-class ROC-AUC can help, but for six-class waste classification:

- reviewers usually care more about confusion matrices and macro-F1.

Do not prioritize ROC curves unless specifically required.

Visualizations You SHOULD Include

1. Side-by-Side Confusion Matrices

Best single figure.

Shows:

- baseline collapse
- CNN structure learning

2. Per-Class F1 Bar Chart

Useful because:

- highlights weak categories
- easy to interpret visually

Example:

- trash class hardest
- cardboard strongest

3. Accuracy vs Macro-F1 Comparison

Small bar chart:

- baseline accuracy
- CNN accuracy
- baseline macro-F1
- CNN macro-F1

This demonstrates:

- why accuracy alone is misleading

4. Training/Validation Curves

Show:

- training loss
- validation loss
- validation accuracy

This helps argue:

- convergence
- no severe overfitting

3. Reviewer Perspective — What Would Get This Rejected?

Here are the biggest methodological flaws.

A. Data Leakage (MOST SERIOUS)

Immediate rejection risk.

Example:

- train/test images from same original photo session
- augmented versions leaking into test set
- duplicates across splits

For image classification, leakage is extremely common.

A reviewer will ask:

“Are train and test images independent?”

If not:

- your 75% may be meaningless.

B. Using Accuracy Only

Huge red flag with imbalanced data.

Your baseline already exposes imbalance:

- majority-class guessing gets 23.67%.

If you only report accuracy:

- reviewer may conclude the CNN mostly predicts “paper”.

You must report:

- macro-F1
- per-class metrics
- confusion matrix

C. No Proper Test Split

Another common rejection reason.

Bad:

- tuning hyperparameters on test set
- reporting best observed run
- no validation separation

Correct:

- train / validation / test
- OR
- cross-validation with held-out folds

D. Single Run, No Variance

If you only report:

“Model accuracy = 75%”

a reviewer may ask:

- what about random seed variation?
- was this cherry-picked?

Better:

- run multiple seeds
- report mean $\pm$ std

Even 3–5 runs helps.

E. Weak Baseline Selection

A reviewer may criticize:

“Majority-class baseline is too weak.”

You should acknowledge this.

Add at least one stronger baseline:

- simple classical ML:
 - SVM on color histograms/HOG
 - Random Forest
- or lightweight pretrained model

Even if they perform worse, this strengthens credibility.

F. No Class Distribution Disclosure

Always show:

- number of samples per class

Otherwise reviewers cannot interpret metrics properly.

Strong Evaluation Structure

A strong evaluation section would look like:

1. Dataset statistics
2. Train/validation/test split
3. Baselines
4. Metrics:
 - accuracy
 - macro-F1
 - balanced accuracy
 - per-class F1
5. Statistical significance:
 - McNemar’s test
6. Visualizations:

- confusion matrices
- class-wise F1 bars
- training curves

7. Error analysis:

- hardest classes
- common confusions

That would be considered solid for a course ML/CV project.



So I should do McNemar's Test. How might I run that using python?

Yes — for your setup, McNemar's test is the cleanest and most defensible choice.

You want to compare:

- Baseline classifier (always predicts "paper")
- CNN classifier
- On the exact same test images

The key idea:

McNemar's test only cares about whether each model was:

- correct
- incorrect

on each sample.

Step 1 — Install Required Package

Bash

Step 2 — Build Paired Correct/Incorrect Results

Suppose you already have:

```
y_true
```

Python

Run

Example:

```
y_true = ["paper", "glass", "metal", ...]
```

Python

Run

Convert predictions into correctness:

```
baseline_correct = [  
    pred == true  
    for pred, true in zip(baseline_preds, y_true)  
]  
  
cnn_correct = [  
    pred == true  
]
```

Python

Run

Step 3 — Build the McNemar Contingency Table

McNemar uses a 2×2 table:

	CNN Correct	CNN Wrong
Baseline Correct	a	b
Baseline Wrong	c	d

Important cells:

- **b** = baseline correct, CNN wrong
- **c** = baseline wrong, CNN correct

If your CNN is better:

- **c** should be much larger than **b**

Code:

```
from statsmodels.stats.contingency_tables import mcnemar

a = b = c = d = 0

for base_ok, cnn_ok in zip(baseline_correct, cnn_correct):

    if base_ok and cnn_ok:
        a += 1
    elif base_ok and not cnn_ok:
        b += 1
    elif not base_ok and cnn_ok:
        c += 1
    else:
        d += 1

table = [[a, b],
         [c, d]]
```

Python

Run

Step 4 — Run McNemar's Test

```
result = mcnemar(table, exact=True)
```

Python

Run

Step 5 — Interpret the Result

Typical interpretation:

```
alpha = 0.05

if result.pvalue < alpha:
    print("Reject null hypothesis")
    print("The CNN significantly outperforms the baseline.")
```

Python

Run

What the Null Hypothesis Means

McNemar tests:

"Both classifiers make errors at the same rate."

A small p-value means:

- the disagreement pattern is asymmetric
- your CNN genuinely improves performance

Important Recommendation

Use:

Python

Run

because:

- your disagreement counts may not be huge
- exact McNemar is more statistically proper for ML projects

Example of What You'd Report in a Paper

We compared the CNN against a majority-class baseline using McNemar's test on paired test predictions. The CNN significantly outperformed the baseline ($p < 0.001$), indicating that the improvement in classification accuracy was unlikely due to chance.

Optional Improvement (Highly Recommended)

Also report:

- accuracy
- macro-F1
- confusion matrix

because McNemar alone only tells:

“better or not better”

It does NOT explain:

- which classes improved
- class imbalance behavior
- minority-class performance

For your project, the combination of:

- McNemar test
- macro-F1
- confusion matrices

is very strong.



So I use the statsmodels package and build a 2x2 contingency table and then run the test using that table and get a p-value

Exactly.

Your workflow is:

1. Get predictions from both models on the SAME test set
2. Convert predictions into:
 - correct
 - incorrect
3. Build the 2×2 contingency table

4. Run McNemar's test
5. Interpret the p-value

The core code is essentially:

```
from statsmodels.stats.contingency_tables import mcnemar

table = [
    [a, b],
    [c, d]
]

result = mcnemar(table, exact=True)
```

Python

Run

Where:

Cell	Meaning
a	both models correct
b	baseline correct, CNN wrong
c	baseline wrong, CNN correct
d	both wrong

The important comparison is:

- c VS b

If:

- CNN fixes many mistakes the baseline makes
- and rarely fails where baseline succeeds

then:

- $c \gg b$
- p-value becomes very small
- significance is established

What You'll Likely See

Since:

- baseline accuracy $\approx 23.67\%$
- CNN accuracy $\approx 75\%$

you will probably get:

- extremely small p-value
- often < 0.001

assuming your test set is reasonably sized.

One Important Reviewer Detail

Do NOT say:

"McNemar proves the CNN is good."

Instead say:

"McNemar's test shows the CNN significantly outperforms the baseline classifier on the paired test set."

That wording is statistically correct.

The Most Common Mistake

People accidentally compare:

- aggregate accuracies

instead of:

- paired per-sample correctness

McNemar REQUIRES paired predictions on identical examples.



I went ahead and implemented that test and received the following results:

Accuracy: 76.88%
Both: 90 | Baseline: 19
CNN: 299 | Neither: 98
P-Value: 6.67753695663154e-66

Those results are extremely strong and internally consistent.

Your contingency table is effectively:

	CNN Correct	CNN Wrong
Baseline Correct	90	19
Baseline Wrong	299	98

So:

- **b = 19** → baseline correct, CNN wrong
- **c = 299** → baseline wrong, CNN correct

That asymmetry is massive.

Because:

- the CNN corrected 299 mistakes the baseline made
- while only introducing 19 new mistakes

the resulting p-value becomes extraordinarily small:

$$p \approx 6.68 \times 10^{-66}$$

which is far below any conventional threshold (0.05, 0.01, 0.001).

How to Write This Properly

You could write:

The trained CNN achieved a test accuracy of 76.88%, substantially outperforming the majority-class baseline accuracy of 23.67%. To determine whether this improvement was statistically significant, McNemar’s test was conducted on paired prediction

outcomes. The test yielded a p-value of 6.68×10^{-66} , indicating that the CNN significantly outperformed the baseline classifier.

What a Reviewer Would Think

A reviewer would likely conclude:

- the improvement is unquestionably real statistically
- the CNN learned meaningful decision boundaries
- performance is not due to random variation

However, they would STILL want:

- macro-F1
- confusion matrix
- per-class recall

≡ ChatGPT ▾

↑ ...

-
- balanced performance
 - minority-class competence
 - practical usefulness

With a large enough dataset, even mediocre improvements can become statistically significant.

Important Interpretation Detail

Your McNemar result shows:

"The CNN is significantly better than the baseline on this test set."

It does NOT show:

- generalization to unseen domains
- robustness
- fairness across classes
- resistance to overfitting

That is why the rest of the evaluation still matters.

Another Useful Statistic You Can Mention

Your disagreement counts alone are compelling:

- CNN uniquely correct: 299
- Baseline uniquely correct: 19

That ratio ($\sim 15.7\times$) is easy for readers to intuitively understand.

One More Recommendation

Since your baseline is extremely weak (majority class), you should explicitly acknowledge that limitation:

While the majority-class baseline provides a useful lower-bound reference, future work should compare against stronger traditional machine learning baselines or lightweight pretrained architectures.

That sentence preempts reviewer criticism.



Ads do not influence the answers you get from ChatGPT. Your chats stay private.
[Learn about ads and personalization](#) >